

Object-oriented programming in C++

Smart pointers

- a key concept in modern C++ -

Dr. Claudia Weis, Faculty of Computer Science

Fakultät für Informatik

UNIVERSITÄT DUISBURG ESSEN

Offen im Denken

RAII – "resource allocation is initialisation"

Fakultät für Informatik

UDE

What it is

(besides an awkward name for a clever yet simple principle)

- Best practice programming technique
- Core concept created by Bjarne Stroustrup (C++ creator) in the late 1980s

How it works

- Acquire in Constructor: Allocates resources (heap memory, file handles, locks) on object creation.
- Release in Destructor: Automatically frees resources when the object leaves scope.

Why it is useful

- No Manual Cleanup: Prevents leaks without manual delete or free().
- Exception Safety: Guarantees automatic cleanup during stack unwinding when exceptions occur.

RAII sounds good – we want it ;-)

Fakultät für Informatik

UDE

- Use modern data types from the C++ standard library!

Examples:

- for file access: `std::ofstream`, `std::ifstream`
- for thread synchronisation: `std::lock_guard`, `std::unique_lock`
- smart pointer: `std::unique_ptr`, `std::shared_ptr`
- standard container `std::vector`, `std::string`, `std::map`

Fakultät für Informatik



Smart pointers help us with the RAII principle for our own user-defined data types

Smart pointers
– the modern C++ concept to apply rules to pointers –

Fakultät für Informatik

UDE

- `unique_ptr` and `shared_ptr` help to simplify memory management
- `unique_ptr`
  - sole owner of data, freed when variable goes out of scope
- `shared_ptr`
  - reference counted, freed when last variable that references this memory goes out of scope
- `weak_ptr`
  - goes together with `shared_ptr` , but does not increase the counter
  - helps to avoid circular dependencies ⟶ probably beyond the scope of this lecture

- Using raw pointers (like in C) is a good idea if you want to learn how valgrind works ;-)
  - Valgrind is a tool to debug memory management bugs/errors

but even if you use valgrind you might not find memory errors because of different compiler behaviour in debug mode ;-)

unique_ptr<type>

- Exclusive ownership: Only one pointer can own the resource at a time.
- No manual memory management: No need to use `new` and `delete`.
- Access via pointer: Use normally (e.g., `->` or *), but…
- Copying is prohibited: The compiler will throw an error if you try to copy it.
- Transfer of ownership: To pass ownership to another variable, use `std::move()`.

Example: a little “game” - overview -

```
#include <iostream>         // required because we want to use std::cout
#include <memory>           // required for smart pointers
#include <string>           // required for the modern std::string

struct Player {
  std::string name; // one member variable
   // constructor and destructor will go here
}; // end of struct Player
void spawnHero() { // code for this function will go here
}
int main() { // as usually, we will test our functions inside the main() function
}
```

a user-defined data type for our game

```
struct Player {
  std::string name; // one member variable

  Player(std::string n) : name(n) {  // constructor, that outputs some info
    std::cout << ">>> " << name << " betritt das Spiel.\n";
  }
  ~Player() { // destructor, that outputs some info
    std::cout << "<<< " << name;
    std::cout << " verlässt das Spiel (automatisch gelöscht!).\n";
  }
};
```

modern string handling allows using umlaute, 'ß', etc.
old-style one had to stick to 7bit ascii or see surprises ;-)

Example how to use a smart pointer
- a function for our game -

Fakultät für Informatik

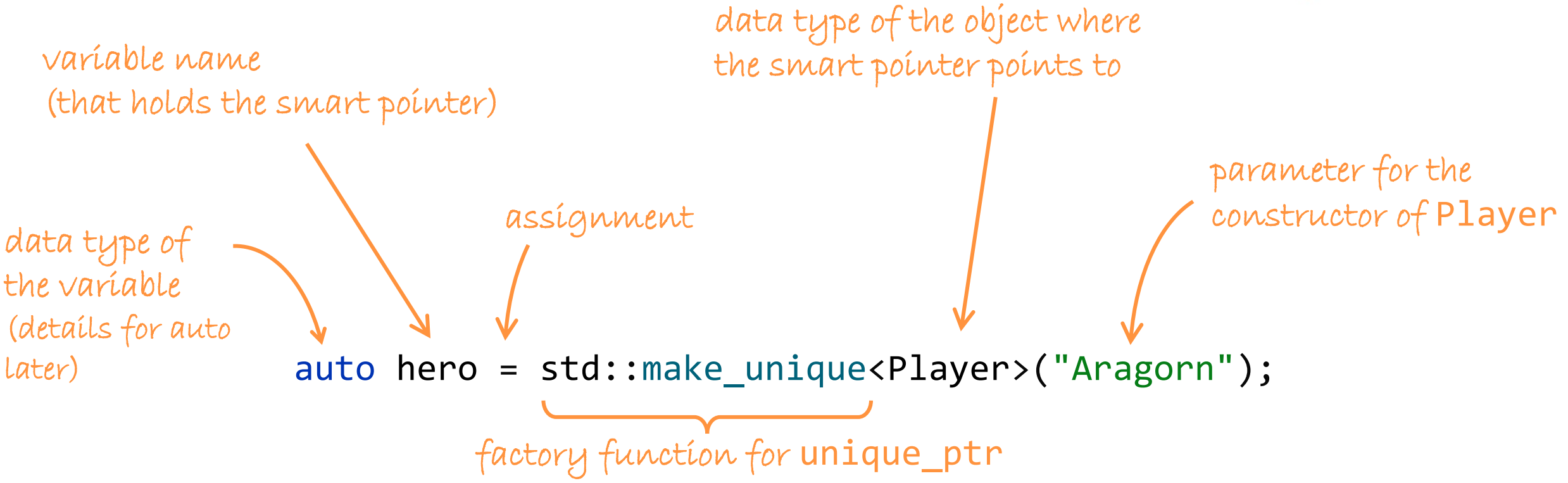
UDE

```
void spawnHero() {
  // Modern creation using std::make_unique (C++14)
  auto hero = std::make_unique<Player>("Aragorn");

  // normal access with -> operator
  // no need to be puzzled about (de-)referencing
  std::cout << "Aktueller Held: " << hero->name << "\n";

  // End of function: hero leaves the scope.
  // The destructor is called AUTOMATICALLY (without explicit delete)
}
```

A detailed look



Smart pointers come with a factory function to prevent programmers from getting directly in touch with raw pointers. (The constructor for unique_ptr requires a raw pointer as a parameter, the factory function takes care of it for us and we do not need to write new.)

“Playing” our game

```
int main() {
  std::cout << "Spiel startet...\n";
  spawnHero();
  std::cout << "Spiel beendet.\n";

  return 0;
}
```

```
> main.out
Spiel startet...
>>> Aragorn betritt das Spiel.
Aktueller Held: Aragorn
<<< Aragorn verlässt das Spiel (automatisch gelöscht!).
Spiel beendet.

Process finished with exit code 0
```

output by spawnHero() function

What is `auto`*?*

- `auto` specifier: instruction for compile-time type deduction, not a distinct data type.
- mechanism: compiler determines the variable's type directly from its initializer value.
- static typing preserved *(important!)*: type binding occurs at compile time (by compiler), not at runtime (unlike Python or JavaScript).

When to use auto*?*

- in general possible: whenever a variable is not only declared but also initialized.
  - examples:

```
auto x = 5; // possible
auto x;     // not possible (will yield compile error)
```

- However it is not always a good idea, if you want to avoid confusion or surprises ;-)
  - examples:

```
auto  x = calculateSomething(); // possible, but what type is x?
float x = calculateSomething(); // expected return type is clear
auto val = 3.14; // default is 'double', but maybe you wanted 'float'?
                 // e.g. for an embedded device with little memory?
```

Best practices when to use `auto`
- example 1: smart pointers and their factory functions -

```
auto hero = std::make_unique<Player>("Aragorn");
```

- The data type of hero becomes directly clear from the right-hand side of the assignment.
- The code line remains readable and avoids redundancy (“copy & paste “), c.f.

```
std::make_unique hero = std::make_unique<Player>("Aragorn");
```

  - remember the DRY principle (Don’t Repeat Yourself)

- If in a future version of our game we want to use another datatype for hero, we have to change only one place, e.g.

```
auto hero = std::make_unique<MasterOfDessaster>("Aragorn");
```

Best practices when to use `auto`
- example 2: Range-based for-loops -

Fakultät für Informatik

UDE

- Remember our pizza example from the beginning of the semester?

```
25  Pizza pizzas[3] = {{8.9, 29.0}, {7.9, 25.0}, {9.9,30.0}};
26      for(auto pizza: pizzas){
27          std::cout << pizza.cm_2_per_euro() << std::endl;
28      }
```

- Note: this example uses a C-style array (which suffers from all the problems C-style arrays automatically have)

We will talk about improvement with respect to modern C++ later...

shared_ptr<type>

- Shared ownership: Multiple pointers can share ownership of the same resource.
- Reference counting: Keeps track of how many pointers currently point to the resource (automatically).
- Automatic cleanup: The resource is destroyed only when the last `shared_ptr` pointing to it is destroyed or reset.
- Modern creation: Use `std::make_shared<Type>(args...)` (just like `make_unique`).
- When to use: When multiple parts of your program need to share access and lifetime of the same object.

Homework: study example with shared_ptr

- Example using `shared_ptr` provided in the OOP-Moodle together with this set of slides.
- Try to understand every line of code with the detailed knowledge of our Game example.
- If you want to learn more (or get stuck), you can
  - ask a friend
    - it's always a good idea to talk to friends ;-)
  - post a question in the OOP Moodle discussion forum
    - try to explain your problem as specific as possible
  - discuss with an AI chat bot (e.g. Google's free of charge AI mode)
    - there is no need to spend money on programming explanations at the level of our lecture
  - ask questions in the exercise session
    - the exercise session is there to support your learning

Best practices for smart pointers

- Avoid `new` and `delete`: Use `std::make_unique` (or `std::make_shared`) instead.
- Use `std::unique_ptr` by default: Only switch to `std::shared_ptr` when it is strictly necessary.
- Do not pass smart pointers to functions without reason: If a function only needs to read or modify the object without changing ownership, simply pass a regular reference (`T&`) or a raw pointer (`T*`) obtained via `.get()`.

Fakultät für Informatik

a little excursion to the history of smart pointers

in case you come across examples using earlier versions of modern C++

C++11: Wrapping manual new with `unique_ptr`.

main.cpp

```
7   #include <memory>
8   #include <cstdint>
9
10  struct Thing {
11      std::unique_ptr<uint8_t[]> memory;
12
13      Thing(int size)
14          : memory(new uint8_t[size]) {
15      }
16  };
17
18  int main() {
19      Thing t(50);
20      Thing t2(50);
21  }
```

```
g++ -O3 -std=c++11 main.cpp
```

C++14: Replacing new with `std::make_unique`.

main.cpp

```
7   #include <memory>
8   #include <cstdint>
9
10  struct Thing {
11      std::unique_ptr<uint8_t[]> memory;
12
13      Thing(int size)
14          : memory(std::make_unique<uint8_t[]>(size)) {
15      }
16  };
17
18  int main() {
19      Thing t(50);
20      Thing t2(50);
21  }
```

```
g++ -O3 -std=c++14 main.cpp
```

C++17: Increasing type safety using `std::byte` instead of `uint8_t`

main.cpp

```
7    #include <memory>
8
9    struct Thing {
10       std::unique_ptr<std::byte[]> memory;
11
12       Thing(int size)
13           : memory(std::make_unique<std::byte[]>(size)) {
14       }
15   };
16
17   int main() {
18       Thing t(50);
19       Thing t2(50);
20   }
```

```
g++ -O3 -std=c++17 main.cpp
```

Semantics: Why `unique_ptr` automatically prevents copying of the enclosing class and forces `std::move`

main.cpp

```cpp
#include <memory>

struct Thing {
    std::unique_ptr<std::byte[]> memory;

    Thing(int size)
        : memory(std::make_unique<std::byte[]>(size)) {
    }
};

int main() {
    Thing t(50);
    Thing t2(50);

    Thing t3 = t; // This line will cause a compilation error
    Thing t3 = std::move(t); // This line is valid
}
```

Fakultät für Informatik

Questions?



"live" in lecture and exercise



discussion forum @ Moodle (preferred, everyone can reply)



by e-mail (use your @uni-due.de email address!)